Manual de instalación

Gestión de Inventario de Componentes Electrónicos

Python, SQLite y bibliotecas necesarias, con todos los comandos y el armado del pendrive

Dato	Valor
Versión del manual	1.1
Fecha	22 de octubre de 2026
Corresponde a	proyecto_completo_PySide6.zip entregado en esa fecha (aplicación PySide6 + pypdfium2 refactorizada por capas, con almacén de documentos por hash SHA-256)
Documentos relacionados	Manual_Inventario (usuario), Manual_Instalacion, Manual_Tecnico

Índice

1. Qué hace falta ... 3

2. Instalar Python 3.13 ... 3

Opción A: instalador de python.org ... 3

Opción B: con winget (Windows 10/11) ... 3

Comprobar la instalación ... 4

3. SQLite ... 4

Herramienta opcional para mirar la base ... 4

4. Preparar la carpeta y un entorno virtual ... 4

5. Instalar las bibliotecas de Python ... 5

Todas juntas ... 5

Una por una ... 5

6. Tesseract OCR (opcional) ... 5

Descargar el instalador de Tesseract ... 6

Instalación automática (sin ventanas) ... 6

Agregar el idioma español ... 6

Dejarlo disponible para la aplicación ... 7

Versión portátil: Tesseract dentro de la carpeta de la aplicación ... 7

7. Verificar la instalación ... 7

8. Ejecutar la aplicación ... 8

Con Visual Studio Code ... 8

9. Instalación resumida ... 8

10. Usar la aplicación desde un pendrive ... 8

11. Generar el ejecutable (.exe) ... 9

Preparación ... 9

Generación ... 9

Una carpeta o un solo archivo ... 10

Probar antes de copiar ... 10

12. Llevar la aplicación al pendrive ... 11

Qué pendrive usar ... 11

Copiar la aplicación ... 11

Lanzarla desde el pendrive ... 11

Uso diario y cuidados ... 11

Actualizar el programa sin perder los datos ... 11

OCR en el pendrive ... 12

13. Cómo queda la estructura de carpetas ... 12

Sin generar el ejecutable (ejecutando con Python) ... 13

Resumen: de cero al pendrive ... 14

14. Problemas frecuentes ... 14

15. Nota para Linux y macOS ... 14

Referencias ... 14

Apéndice A. Árboles de directorio: sin ejecutable y con ejecutable en el pendrive ... 16

D.1 Sin ejecutable: carpeta de la aplicación en la computadora de desarrollo ... 16

D.2 Sin ejecutable, llevado al pendrive ... 16

D.3 Con ejecutable, en el pendrive ... 16

D.4 Qué cambia entre los tres casos ... 17

1. Qué hace falta

La aplicación (archivo de arranque app_inventario.py) necesita tres cosas: el lenguaje Python, una base de datos SQLite y siete bibliotecas de Python. SQLite viene incluida con Python, por lo que no hay que instalarla aparte. Un octavo componente, el programa Tesseract, es opcional y solo sirve para reconocer texto en páginas escaneadas.

Los comandos de este manual están escritos para Windows 10 u 11 (64 bits), que es el entorno de uso de la aplicación, y se ejecutan en PowerShell o en el Símbolo del sistema (cmd). Al final hay una nota para Linux y macOS.

Componente	Se instala con	Se importa como	Para qué se usa	¿Obligatorio?
Python 3.13	Instalador de python.org o winget	—	Lenguaje de toda la aplicación.	Sí
SQLite	Viene con Python (módulo sqlite3)	sqlite3	Base de datos del inventario.	Sí (incluida)
PySide6	pip install PySide6	PySide6	Interfaz gráfica (Qt for Python).	Sí
pypdfium2	pip install pypdfium2	pypdfium2	Visor de PDF y recortes (motor PDFium).	Sí
ReportLab	pip install reportlab	reportlab	PDF de etiquetas.	Sí
qrcode	pip install "qrcode[pil]"	qrcode	Código QR.	Sí
Pillow	pip install pillow	PIL	Imágenes (lo usa qrcode).	Sí
openpyxl	pip install openpyxl	openpyxl	Planilla de Excel del capital.	Para exportar el capital
pytesseract	pip install pytesseract	pytesseract	Puente de Python hacia Tesseract.	Opcional (OCR)
Tesseract OCR	Instalador de UB Mannheim o winget	(programa aparte)	Reconoce texto en páginas escaneadas.	Opcional (OCR)

Nota: El archivo requirements.txt que acompaña a la aplicación ya contiene las siete bibliotecas de Python (PySide6, pypdfium2, reportlab, qrcode[pil], pillow, pytesseract y openpyxl), de modo que un único comando las instala todas.

Nota: La aplicación usa PySide6 (el «Qt for Python» oficial de The Qt Company), no PyQt5 ni PyQt6. Son bibliotecas distintas y no son intercambiables: el código importa PySide6 en todos sus módulos. Si en el equipo hay instalado PyQt5 o PyQt6, pueden convivir con PySide6 sin molestarse. Para ver cuál está instalada: python -m pip list | findstr /i "pyside pyqt" (en PowerShell: python -m pip list | Select-String -Pattern "pyside|pyqt").

2. Instalar Python 3.13

Opción A: instalador de python.org

1. Entrar a https://www.python.org/downloads/ y descargar el instalador de Windows (64 bits) de Python 3.13.

2. Ejecutarlo y, en la primera pantalla, tildar «Add python.exe to PATH» antes de pulsar «Install Now». Es el paso que más se olvida y evita el error «python no se reconoce como un comando».

3. Terminar la instalación y cerrar las ventanas de PowerShell o cmd que estuvieran abiertas, para que tomen la configuración nueva.

Opción B: con winget (Windows 10/11)

Si el equipo tiene winget, en PowerShell:

```
winget install Python.Python.3.13
```

Después de instalar, abrir una ventana nueva de PowerShell.

Comprobar la instalación

```
python --version
python -m pip --version
```

El primer comando debe mostrar «Python 3.13.x». Si Windows abre la tienda de aplicaciones o no reconoce el comando, probar con el lanzador «py»: py --version (o py -3.13 --version si hay varias versiones). En ese caso, usar «py» en lugar de «python» en el resto del manual.

Conviene actualizar pip, el instalador de paquetes:

```
python -m pip install --upgrade pip
```

3. SQLite

SQLite no se instala por separado: el módulo sqlite3 forma parte de Python 3.13 y es el que usa la aplicación. La base de datos es un único archivo, inventario_electronica.sqlite3, que el programa crea solo la primera vez que se abre, dentro de la carpeta Base_Inventario.

Para ver qué versión de SQLite trae la instalación de Python:

```
python -c "import sqlite3; print(sqlite3.sqlite_version)"
```

Herramienta opcional para mirar la base

No es necesaria para usar la aplicación, pero permite abrir el archivo .sqlite3 y ver las tablas. La más usada es DB Browser for SQLite, que se instala con winget:

```
winget install DBBrowserForSQLite.DBBrowserForSQLite
```

También se puede descargar de https://sqlitebrowser.org. Conviene abrir la base solo para consultar y con la aplicación cerrada; los cambios hechos a mano pueden dejar datos inconsistentes.

4. Preparar la carpeta y un entorno virtual

Descomprimir la aplicación en una carpeta (por ejemplo C:\Inventario o una carpeta del pendrive). Debe quedar junto a app_inventario.py el archivo requirements.txt y las carpetas del programa (interfaz y los demás módulos).

> **Nota:** El entorno virtual es recomendable pero NO es obligatorio. La aplicación funciona igual si las bibliotecas se instalan directamente en Python. Si se prefiere no crearlo, se omiten los comandos de este capítulo y, en el capítulo 5, se ejecutan los mismos comandos pip sin haber activado nada (opcionalmente con --user, que instala solo para el usuario actual: python -m pip install --user -r requirements.txt). Tampoco es necesario para generar o usar el ejecutable en otros equipos, aunque sí es útil en la computadora de desarrollo.

	Con entorno virtual	Sin entorno virtual
Ventaja	Las bibliotecas de la aplicación quedan aisladas; no chocan con otros programas de Python; se puede borrar y recrear fácilmente.	Es más simple: menos pasos y nada que activar.
Inconveniente	Hay que activarlo cada vez que se abre una ventana de comandos.	Las bibliotecas se mezclan con las de otros programas de Python del equipo y sus versiones pueden chocar.
Conviene cuando	Se desarrolla o se generan ejecutables, o el equipo usa Python para otras cosas.	El equipo es de uso exclusivo para esta aplicación, o solo se quiere ejecutarla.

Para crear el entorno virtual, desde la carpeta de la aplicación:

```
cd C:\Inventario
python -m venv venv
```

Para activarlo:

Programa	Comando de activación
PowerShell	`.\venv\Scripts\Activate.ps1`
Símbolo del sistema (cmd)	`venv\Scripts\activate.bat`

Cuando está activo, el indicador de la línea de comandos empieza con «(venv)». Hay que activarlo cada vez que se abre una ventana nueva, antes de instalar o de ejecutar la aplicación.

> **Nota:** Si PowerShell responde que la ejecución de scripts está deshabilitada, se habilita para el usuario actual con: Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned. También se puede usar cmd, que no tiene esa restricción.

5. Instalar las bibliotecas de Python

Todas juntas

Con el entorno activado (si se creó uno) y desde la carpeta de la aplicación:

```
python -m pip install -r requirements.txt
```

Una por una

Si se prefiere instalarlas individualmente, o si alguna falla y hay que repetirla:

```
python -m pip install PySide6
python -m pip install pypdfium2
python -m pip install reportlab
python -m pip install "qrcode[pil]"
python -m pip install pillow
python -m pip install openpyxl
python -m pip install pytesseract
```

Qué hace cada una, para saber cuál falta según el síntoma:

Si falta…	Se nota porque…
PySide6	La aplicación no abre: error «No module named 'PySide6'».
pypdfium2	Falla al abrir un datasheet, un tutorial o la previsualización de etiquetas («No module named 'pypdfium2'»).
reportlab	Falla al generar el PDF de etiquetas.
qrcode y pillow	No aparece el código QR.
openpyxl	El botón «Exportar a Excel» del capital avisa que falta la biblioteca.
pytesseract / Tesseract	El clic derecho del visor de PDF avisa que el OCR no está disponible; el resto funciona normalmente.

6. Tesseract OCR (opcional)

El reconocimiento de texto sobre páginas que son imágenes necesita dos cosas: la biblioteca pytesseract (ya instalada en el paso anterior) y el programa Tesseract. Sin él, la aplicación funciona igual y solo se pierde esa función.

1. Instalar Tesseract. La forma más simple es con winget:

```
winget install UB-Mannheim.TesseractOCR
```

También puede descargarse el instalador de Windows de https://github.com/UB-Mannheim/tesseract/wiki.

2. Si se usa el instalador gráfico, en la pantalla de componentes desplegar «Additional language data» y tildar «Spanish» (además de English), para que reconozca textos en español. La aplicación elige sola la mejor combinación de idiomas instalada (inglés más español, si están).

3. Reiniciar la aplicación. Esta busca Tesseract en el PATH y en las carpetas habituales de Windows (C:\Program Files\Tesseract-OCR, C:\Program Files (x86)\Tesseract-OCR y la carpeta de programas del usuario).

Si Tesseract quedó en otra carpeta, se le indica la ruta con la variable de entorno TESSERACT_CMD (en PowerShell, antes de ejecutar la aplicación):

```
$env:TESSERACT_CMD = "D:\Herramientas\Tesseract-OCR\tesseract.exe"
```

Para comprobar que quedó instalado:

```
tesseract --version
tesseract --list-langs
```

Descargar el instalador de Tesseract

Tesseract para Windows no lo publica el proyecto original sino la Universidad de Mannheim (UB Mannheim). Pasos para descargarlo a mano:

1. Abrir en el navegador https://github.com/UB-Mannheim/tesseract/wiki

2. Buscar la tabla de instaladores y descargar el de 64 bits: un archivo con un nombre como tesseract-ocr-w64-setup-5.4.0.20240606.exe (la numeración cambia con cada versión; elegir la más reciente).

3. Guardarlo, por ejemplo, en la carpeta Descargas o en el pendrive (puede quedar guardado como instalador para usarlo en otros equipos).

Para descargarlo desde la línea de comandos, con winget (descarga e instala en el equipo):

```
winget install UB-Mannheim.TesseractOCR
```

Para solo descargar el instalador sin instalarlo (queda en la carpeta indicada; requiere una versión de winget reciente):

```
winget download UB-Mannheim.TesseractOCR -d C:\Instaladores
```

Instalación automática (sin ventanas)

El instalador de UB Mannheim es del tipo NSIS y acepta el parámetro /S (silencioso): se instala sin mostrar pantallas ni pedir nada. En PowerShell, estando en la carpeta donde está el instalador (Start-Process espera a que termine):

```
Start-Process -Wait .\tesseract-ocr-w64-setup-5.4.0.20240606.exe -ArgumentList "/S"
```

Con /S se instala en la carpeta habitual (C:\Program Files\Tesseract-OCR), que la aplicación ya busca sola; por eso, después de instalarlo basta con reiniciar la aplicación. Si se necesita otra carpeta, el instalador NSIS acepta además /D=carpeta, que debe escribirse al final y sin comillas (ver el apartado siguiente).

> **Nota:** En modo silencioso el instalador pone solo los componentes por defecto (inglés). El idioma español no se tilda solo: hay que agregarlo, como se explica en el apartado «Agregar el idioma español». Los parámetros /S y el nombre del instalador se tomaron de la ficha de winget (ver Referencias); /D= es una convención general de NSIS y no se pudo probar con este instalador.

Agregar el idioma español

Si se instaló en forma silenciosa, o no se tildó «Spanish» en el instalador gráfico, se descarga el archivo de idioma y se copia dentro de la carpeta tessdata de Tesseract. El archivo es spa.traineddata, del repositorio oficial de datos de Tesseract (versión rápida, de poco tamaño):

```
Invoke-WebRequest -Uri
"https://github.com/tesseract-ocr/tessdata_fast/raw/main/spa.traineddata" `
  -OutFile "C:\Program Files\Tesseract-OCR\tessdata\spa.traineddata"
```

Copiar en C:\Program Files exige permisos de administrador: abrir PowerShell como administrador, o descargar el archivo y copiarlo a mano. Para comprobar que quedó, tesseract --list-langs debe listar «spa».

Dejarlo disponible para la aplicación

La aplicación busca Tesseract, en este orden:

1. La variable de entorno TESSERACT_CMD, si está definida.
2. Una carpeta Tesseract-OCR junto a la aplicación (junto a app_inventario.py o a Inventario.exe). Esta es la forma portátil.
3. El PATH de Windows.
4. Las carpetas habituales: C:\Program Files\Tesseract-OCR, C:\Program Files (x86)\Tesseract-OCR y la carpeta de programas del usuario.

No hay que configurar nada más: si Tesseract está en alguno de esos lugares, la aplicación lo encuentra al intentar usar el OCR, y lo ejecuta ella misma en segundo plano cada vez que se usa «Modo Texto» o el menú de OCR del visor de PDF. El usuario no lo abre nunca. Basta reiniciar la aplicación después de instalarlo.

Versión portátil: Tesseract dentro de la carpeta de la aplicación

Para que el OCR funcione en cualquier equipo sin instalar nada (por ejemplo, desde el pendrive), se coloca Tesseract en una carpeta Tesseract-OCR junto a la aplicación. Hay dos maneras:

Manera A, instalar directamente en esa carpeta (con el instalador silencioso y /D=, que debe ir al final):

```
Start-Process -Wait .\tesseract-ocr-w64-setup-5.4.0.20240606.exe -ArgumentList "/S",
"/D=E:\Inventario\Tesseract-OCR"
```

Manera B, copiar una instalación existente: instalar Tesseract en un equipo (de cualquier forma) y copiar la carpeta completa C:\Program Files\Tesseract-OCR al pendrive, dentro de la carpeta de la aplicación, con el nombre Tesseract-OCR:

```
Copy-Item "C:\Program Files\Tesseract-OCR" E:\Inventario\Tesseract-OCR -Recurse
```

En ambas maneras, el archivo tesseract.exe debe quedar en E:\Inventario\Tesseract-OCR\tesseract.exe, y la carpeta tessdata (con eng.traineddata y spa.traineddata) dentro de Tesseract-OCR. Para comprobarlo antes de abrir la aplicación:

```
E:\Inventario\Tesseract-OCR\tesseract.exe --list-langs
```

> **Nota:** Esta búsqueda en la carpeta de la aplicación es una modificación del módulo ocr.py hecha para esta guía. Está incluida en el proyecto entregado junto a este manual (proyecto_completo_PySide6.zip); si se usa una copia anterior del programa, hay que reemplazar ocr.py. En el ejecutable, hay que volver a generarlo después de reemplazarlo. Se probó la lógica de búsqueda con una carpeta de prueba, pero no con Tesseract real instalado en Windows.

7. Verificar la instalación

Estos comandos muestran la versión de cada pieza; si alguno da error, esa pieza no quedó bien instalada:

```
python -c "import sys, sqlite3; print('Python', sys.version.split()[0]);
print('SQLite', sqlite3.sqlite_version)"
python -c "import PySide6; print('PySide6', PySide6.__version__)"
python -c "import pypdfium2; print('pypdfium2', pypdfium2.version.PYPDFIUM_INFO)"
python -c "import reportlab; print('ReportLab', reportlab.Version)"
python -c "import qrcode, PIL; print('Pillow', PIL.__version__)"
python -c "import openpyxl; print('openpyxl', openpyxl.__version__)"
python -c "import pytesseract; print('pytesseract', pytesseract.__version__)"
```

Para ver todas las bibliotecas instaladas con sus versiones, y guardarlas en un archivo como registro de la instalación que funciona:

```
python -m pip list
python -m pip freeze > versiones_instaladas.txt
```

Guardar ese archivo es útil: si en otro equipo se quiere reproducir exactamente la misma instalación, alcanza con ejecutar python -m pip install -r versiones_instaladas.txt.

8. Ejecutar la aplicación

Con el entorno virtual activado (si se creó), desde la carpeta de la aplicación:

```
python app_inventario.py
```

La primera vez, el programa crea las carpetas Base_Inventario, adjuntos y Etiquetas junto al archivo app_inventario.py, y la base de datos con el catálogo y las listas iniciales. No hay que crear nada a mano.

Con Visual Studio Code

1. Abrir la carpeta de la aplicación (Archivo › Abrir carpeta).
2. Elegir el intérprete: Ctrl+Mayús+P, escribir «Python: Select Interpreter» y elegir el que está dentro de la carpeta venv (venv\Scripts\python.exe).
3. Abrir app_inventario.py y ejecutar con el botón de ejecución (▶️) o desde la terminal integrada con python app_inventario.py.

9. Instalación resumida

Todos los comandos en orden, para un equipo nuevo con Windows 10/11 (una ventana de PowerShell, desde la carpeta de la aplicación). Las dos líneas del entorno virtual (venv y Activate) pueden omitirse:

```
winget install Python.Python.3.13
# cerrar y volver a abrir PowerShell
cd C:\Inventario
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
winget install UB-Mannheim.TesseractOCR      # opcional (OCR)
python app_inventario.py
```

10. Usar la aplicación desde un pendrive

La aplicación guarda todos sus datos dentro de su propia carpeta, por lo que puede copiarse a un pendrive. Lo que no viaja con ella es Python: debe haber una instalación en el equipo donde se ejecute. Hay tres maneras de resolverlo:

Opción	Cómo	Conviene cuando…
Instalar en cada equipo	Seguir los pasos 2 a 6 en cada computadora; la carpeta de la aplicación viaja en el pendrive y se ejecuta con python app_inventario.py.	Se usa en pocos equipos fijos. Es la más simple y confiable.
Python portátil en el pendrive	Guardar en el pendrive una distribución de Python que no requiera instalación (por ejemplo, WinPython) e instalar allí las bibliotecas con su pip. Se ejecuta con el python.exe del pendrive.	Se usa en equipos donde no se puede instalar software. Hay que comprobar con «pip list» qué bibliotecas incluye y completar las que falten.
Programa compilado (.exe)	Empaquetar la aplicación con una herramienta como PyInstaller. La aplicación ya reconoce este caso y toma las carpetas del lugar donde está el ejecutable.	Se quiere usarla en equipos sin Python. Es la opción que se desarrolla en los capítulos 11 a 13.

> **Nota:** El entorno virtual (carpeta venv) no es portátil: guarda rutas absolutas del equipo donde se creó y deja de funcionar si cambia la letra de unidad del pendrive o se usa otra computadora. Para llevar la aplicación, copiar la carpeta de la aplicación sin la carpeta venv y crear un entorno nuevo en cada equipo.

Recordar cerrar la aplicación antes de retirar el pendrive y hacer copias periódicas de las carpetas Base_Inventario y adjuntos.

11. Generar el ejecutable (.exe)

Un ejecutable reúne la aplicación, Python y todas las bibliotecas en una carpeta, de modo que el equipo donde se use no necesita tener Python instalado. Se genera con PyInstaller.

> **Nota:** El ejecutable debe generarse en Windows para que funcione en Windows: PyInstaller no compila de un sistema a otro. Se genera una sola vez en una computadora de desarrollo (la que ya tiene Python y las bibliotecas) y el resultado se copia a los demás equipos. Cada vez que se modifique el código hay que volver a generarlo.

Preparación

Con el entorno virtual activado (capítulo 4) y las bibliotecas instaladas (capítulo 5), instalar PyInstaller en el mismo entorno:

```
python -m pip install pyinstaller
python -m PyInstaller --version
```

Se necesita una versión reciente de PyInstaller para que sea compatible con Python 3.13; si la instalación indicara incompatibilidad, actualizar con python -m pip install --upgrade pyinstaller.

Generación

Desde la carpeta de la aplicación (donde está app_inventario.py):

```
python -m PyInstaller --noconfirm --clean --windowed --name Inventario
app_inventario.py
```

El comando usa cuatro opciones. Hay otras que pueden hacer falta; esta es la guía de las más importantes, con su forma corta y si la aplicación las necesita:

Opción (forma corta)	Qué hace	¿En esta aplicación?
--onedir (-D)	Genera una carpeta con el ejecutable y las bibliotecas separadas. Es el formato por defecto: si no se indica nada, se usa este. Compila y arranca más rápido.	Sí (por defecto). Recomendado también para el pendrive.
--onefile (-F)	Empaqueta todo en un único .exe. Facilita repartirlo, pero en cada arranque extrae los archivos a una carpeta temporal y tarda unos segundos más.	Opcional (ver tabla siguiente).
--windowed (-w)	En Windows no abre la ventana negra de comandos al ejecutar. Imprescindible en aplicaciones gráficas como esta.	Sí.
--noconsole	Es sinónimo exacto de --windowed: hace lo mismo. Se usa una u otra, no ambas.	No hace falta (ya se usa --windowed).

Opción (forma corta)	Qué hace	¿En esta aplicación?
--name nombre (-n)	Nombre del ejecutable y de la carpeta resultante. Ejemplo: --name InventarioComponentes. Si se omite, toma el nombre del script (app_inventario).	Sí: Inventario.
--icon=ruta.ico (-i)	Icono del ejecutable (.ico en Windows, .icns en macOS) en lugar del genérico.	Opcional.
--clean	Borra la caché de PyInstaller y los temporales antes de generar. Evita errores raros por restos de generaciones anteriores.	Recomendado.
--noconfirm (-y)	Sobrescribe la carpeta de salida (dist) sin pedir confirmación.	Sí.
--hidden-import módulo	Fuerza la inclusión de un módulo que PyInstaller no detectó. Se usa si aparece «No module named …».	Solo si hace falta.
--add-data origen;destino	Incluye dentro del paquete archivos o carpetas extra (imágenes, plantillas, bases iniciales). En Windows el separador es punto y coma (--add-data "iconos;iconos"); en Linux y macOS, dos puntos.	No debe usarse para los datos (ver advertencia).

Nota: --add-data NO sirve para la base de datos ni para los adjuntos de esta aplicación. Los archivos agregados así quedan dentro del paquete (en _internal, o en una carpeta temporal con --onefile): son de solo lectura en la práctica y, con --onefile, se pierden al cerrar. Los datos de la aplicación (Base_Inventario, adjuntos y Etiquetas) deben quedar fuera, junto a Inventario.exe, y los crea el programa solo. --add-data solo se usaría si más adelante se agregaran recursos fijos de la aplicación, como imágenes o iconos que se carguen desde archivos.

Con la forma corta, el comando recomendado equivale a:

```
python -m PyInstaller -y --clean -D -w -n Inventario app_inventario.py
```

La generación demora uno o dos minutos. Al terminar, el resultado queda en la carpeta dist\Inventario. Las carpetas build y el archivo Inventario.spec son auxiliares de la generación y no se llevan al pendrive.

Una carpeta o un solo archivo

PyInstaller ofrece dos formatos. Para esta aplicación se recomienda el primero (el comando de arriba):

Formato	Comando	Ventajas	Inconvenientes
Carpeta (recomendado)	(el comando de arriba; --onedir es el valor por defecto)	Arranca rápido; menos falsas alarmas del antivirus; fácil de revisar.	Son muchos archivos; se copia la carpeta completa.
Un solo archivo	agregar --onefile	Un único Inventario.exe.	Cada arranque desempaqueta el contenido en una carpeta temporal: tarda más, es más lento desde un pendrive y el antivirus lo sospecha con más frecuencia.

En ambos casos la aplicación guarda sus datos junto al archivo Inventario.exe (no en la carpeta temporal), porque su módulo de configuración toma como carpeta base la del ejecutable.

Probar antes de copiar

```
dist\Inventario\Inventario.exe
```

Conviene abrir el ejecutable desde esa carpeta, ingresar un componente de prueba, abrir un PDF y generar una etiqueta, para comprobar que se incluyeron todas las bibliotecas. Esa prueba crea las carpetas de datos dentro de dist\Inventario; antes de copiar el programa al pendrive hay que borrarlas (Base_Inventario, adjuntos y Etiquetas), o se llevaría la base de prueba.

Nota: Esta guía no pudo comprobarse generando el ejecutable en este proyecto, porque la aplicación se desarrolló sin acceso a un Windows con PySide6. Si PyInstaller no incluyera alguna biblioteca, la aplicación mostrará «No module named …» al abrirse; en ese caso se agrega con la opción --hidden-import nombre_del_modulo y se vuelve a generar. Para publicar el ejecutable conviene mantener la carpeta (modo --onedir, el predeterminado): PySide6 usa Qt bajo licencia LGPL v3, que exige que el usuario pueda reemplazar las bibliotecas de Qt, algo que el modo de un solo archivo (--onefile) dificulta.

12. Llevar la aplicación al pendrive

Qué pendrive usar

- Formatear el pendrive como exFAT o NTFS. El formato FAT32 no admite archivos de más de 4 GB, y la carpeta de adjuntos (manuales y hojas de datos) puede crecer mucho.
- Conviene un pendrive de buena velocidad: la base de datos y los PDF se leen y escriben directamente desde él.

Copiar la aplicación

1. En el pendrive, crear la carpeta que contendrá todo, por ejemplo E:\Inventario (la letra de unidad puede ser otra).

2. Copiar dentro de ella el contenido de dist\Inventario (el archivo Inventario.exe y la carpeta _internal, con todo su contenido). No copiar build, venv ni el archivo .spec.

3. Si ya existe un inventario cargado que se quiere llevar, copiar también, desde la carpeta de la aplicación de desarrollo, las carpetas Base_Inventario y adjuntos completas (con la base y los documentos). Si el inventario va a empezar vacío, no copiar nada más.

```
# ejemplo desde PowerShell, con el pendrive como E:
New-Item -ItemType Directory E:\Inventario
Copy-Item .\dist\Inventario\* E:\Inventario\ -Recurse
# solo si se lleva un inventario existente:
Copy-Item C:\Inventario\Base_Inventario E:\Inventario\ -Recurse
Copy-Item C:\Inventario\adjuntos E:\Inventario\ -Recurse
```

Lanzarla desde el pendrive

1. Conectar el pendrive en cualquier computadora con Windows (64 bits).

2. Abrir la carpeta E:\Inventario y hacer doble clic en Inventario.exe. No hace falta instalar nada.

3. La primera vez, si faltaran las carpetas Base_Inventario, adjuntos o Etiquetas, la aplicación las crea sola junto al ejecutable.

Si Windows muestra el aviso «Windows protegió su PC» (SmartScreen), es porque el ejecutable no tiene firma digital: se pulsa «Más información» y luego «Ejecutar de todas formas». Algunos antivirus también pueden bloquearlo la primera vez; si eso ocurre, se agrega la carpeta a las excepciones.

Uso diario y cuidados

- Cerrar la aplicación antes de retirar el pendrive y retirarlo con «Expulsar», para no dañar la base de datos.
- No abrir la aplicación desde dos computadoras a la vez con el mismo pendrive, ni dos copias del programa sobre la misma base.
- Hacer copias de seguridad periódicas: copiar las carpetas Base_Inventario y adjuntos a otro disco. Con esas dos carpetas se recupera todo el inventario.
- No renombrar ni mover archivos dentro de adjuntos: la base los referencia por su ubicación y su hash.
- El pendrive no debe estar protegido contra escritura, porque la aplicación guarda allí la base, los adjuntos y las etiquetas.

Actualizar el programa sin perder los datos

Cuando se genere una versión nueva del ejecutable, se reemplazan en el pendrive solo el archivo Inventario.exe y la carpeta _internal por los nuevos. Las carpetas Base_Inventario, adjuntos y Etiquetas no se tocan: contienen los datos. Antes de actualizar conviene hacer una copia de esas carpetas.

OCR en el pendrive

El programa Tesseract (capítulo 6) no queda incluido dentro del ejecutable, pero puede viajar en el pendrive como una carpeta Tesseract-OCR junto a Inventario.exe. La aplicación la detecta sola, sin instalar nada en el equipo. Las dos maneras de dejarla ahí (instalarla directamente con /D= o copiar una instalación existente) están en el capítulo 6, apartado «Versión portátil». Si no se lleva esa carpeta y el equipo no tiene Tesseract, solo se pierde el OCR sobre escaneos; el resto de la aplicación funciona igual.

13. Cómo queda la estructura de carpetas

Hay dos casos: con ejecutable (primer árbol) y sin ejecutable (más abajo, en este mismo capítulo). Esta es la estructura del pendrive con el ejecutable, después de copiar la aplicación y de usarla. Las carpetas marcadas con «datos» son las que crea y mantiene la aplicación y las que hay que respaldar.

```
E:\
└── Inventario\
    ├── Inventario.exe                ← el programa (doble clic para abrir)
    ├── _internal\                    ← Python y bibliotecas (no tocar)
    │   ├── python313.dll
    │   ├── base_library.zip
    │   ├── PySide6\ ... (interfaz gráfica; incluye las bibliotecas de Qt)
    │   ├── pypdfium2\ , pypdfium2_raw\ ... (visor de PDF)
    │   ├── reportlab\ , PIL\ , openpyxl\ ...
    │   └── (otros archivos de apoyo)
    ├── Tesseract-OCR\                ← opcional (OCR portátil)
    │   ├── tesseract.exe
    │   └── tessdata\ (eng, spa …)
    ├── Base_Inventario\              ← datos
    │   └── inventario_electronica.sqlite3   (la base de datos)
    ├── adjuntos\                     ← datos
    │   ├── almacen\                  (documentos, por hash SHA-256)
    │   │   ├── 3f\
    │   │   │   └── 3fa9c1…e7b2.pdf
    │   │   ├── a0\
    │   │   │   └── a04d58…19c0.png
    │   │   └── …                     (una subcarpeta por los 2 primeros
    │   │                              caracteres del hash)
    │   ├── tmp\                      (temporales; se vacía sola)
    │   └── papelera\                 (archivos ya sin uso)
    └── Etiquetas\                    ← datos
        └── etiquetas.pdf             (PDF de etiquetas generado)
```

Elemento	Qué contiene	¿Se respalda?	¿Se reemplaza al actualizar?
Inventario.exe	El programa.	No	Sí
_internal	Python y las bibliotecas (en versiones antiguas de PyInstaller sus archivos están junto al .exe).	No	Sí
Tesseract-OCR (opcional)	Programa de OCR con sus idiomas.	No	Sí
Base_Inventario	La base SQLite con productos, parámetros, catálogos y cotizaciones.	Sí, imprescindible	No
adjuntos\almacen	Los documentos, con nombre igual a su hash.	Sí, imprescindible	No
adjuntos\tmp y papelera	Temporales y archivos sin uso.	No es necesario	No
Etiquetas	PDF de etiquetas generados.	Opcional	No

Nota: El nombre exacto del PDF dentro de Etiquetas depende de lo que indique el programa al generarlo. Los nombres de las subcarpetas de almacen (3f, a0…) son ejemplos: son los dos primeros caracteres del hash de cada documento, por lo que cada instalación tendrá los suyos.

En la computadora de desarrollo, la carpeta de la aplicación tiene una estructura distinta: contiene los archivos de código (app_inventario.py, config.py, db.py y los demás), la carpeta interfaz, el requirements.txt, el entorno virtual venv y, tras generar el ejecutable, las carpetas build y dist. De todo eso, al pendrive solo viaja el contenido de dist\Inventario y, si corresponde, las carpetas de datos.

Sin generar el ejecutable (ejecutando con Python)

Si no se genera el ejecutable, la aplicación se usa como programa de Python: no existen las carpetas build, dist ni _internal, y los datos se guardan junto a app_inventario.py. Esta es la estructura después de instalar y de usar la aplicación (en el disco del equipo o en el pendrive):

```
C:\Inventario\                          (o E:\Inventario\ en el pendrive)
├── app_inventario.py                ← archivo de arranque (python app_inventario.py)
├── config.py                        ← rutas de la aplicación
├── catalogo.py                      ← taxonomía y listas iniciales
├── db.py                            ← acceso a SQLite (todo el SQL)
├── almacenamiento.py                ← guardado de documentos por hash
├── utilidades.py
├── ocr.py                           ← reconocimiento de texto (Tesseract)
├── pdf_motor.py                     ← lectura de PDF (pypdfium2)
├── capital.py                       ← cálculo y Excel del capital
├── migrar_a_hash.py                 ← migración de adjuntos antiguos
├── requirements.txt                 ← lista de bibliotecas
├── interfaz\                        ← ventanas y diálogos
│   ├── __init__.py
│   ├── ventana_principal.py
│   ├── dialogo_componente.py
│   ├── dialogo_parametros.py
│   ├── dialogo_catalogos.py
│   ├── dialogo_capital.py
│   ├── dialogo_etiquetas.py
│   ├── arbol_parametros.py
│   ├── editor_imagen.py
│   ├── visor_pdf.py
│   └── widgets.py
├── Tesseract-OCR\                   ← opcional (OCR portátil, ver capítulo 6)
├── venv\                            ← entorno virtual (opcional, ver capítulo 4)
├── __pycache__\                     ← la crea Python solo (no tocar)
├── Base_Inventario\                 ← datos
│   └── inventario_electronica.sqlite3
├── adjuntos\                        ← datos
│   ├── almacen\ (por hash)  ├── tmp\  └── papelera\
└── Etiquetas\                       ← datos (PDF de etiquetas)
```

La carpeta interfaz puede contener también otros archivos con nombres como dialogo_componente.etapa7.py: son copias de respaldo de etapas anteriores del desarrollo, no se usan y pueden borrarse. Lo imprescindible para ejecutar son los archivos .py de la lista de arriba y la carpeta interfaz.

Concepto	Con ejecutable	Sin ejecutable
Equipo donde se usa	No necesita Python ni bibliotecas.	Necesita Python 3.13 y las bibliotecas (capítulos 2 a 5).
Qué se copia al pendrive	El contenido de dist\Inventario (Inventario.exe y _internal).	La carpeta de la aplicación: archivos .py, interfaz y requirements.txt, sin venv.
Cómo se abre	Doble clic en Inventario.exe.	python app_inventario.py (o py app_inventario.py) desde una ventana de comandos.
Dónde quedan los datos	Junto a Inventario.exe.	Junto a app_inventario.py.
Cambiar el código	Hay que volver a generar el ejecutable.	Se edita el .py y se vuelve a abrir.

Resumen: de cero al pendrive

```
cd C:\Inventario
.\venv\Scripts\Activate.ps1
python -m pip install pyinstaller
python -m PyInstaller --noconfirm --clean --windowed --name Inventario
app_inventario.py
# probar dist\Inventario\Inventario.exe y borrar las carpetas de datos de prueba
New-Item -ItemType Directory E:\Inventario
Copy-Item .\dist\Inventario\* E:\Inventario\ -Recurse
E:\Inventario\Inventario.exe
```

14. Problemas frecuentes

Síntoma	Causa probable y solución
«python no se reconoce como un comando»	Python no está en el PATH. Reinstalar tildando «Add python.exe to PATH», o usar el lanzador: py app_inventario.py. Cerrar y volver a abrir la ventana de comandos.
«pip no se reconoce»	Usar siempre la forma python -m pip install …
PowerShell no deja activar el entorno virtual	Ejecutar Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned, o activar desde cmd con venv\Scripts\activate.bat.
«No module named 'PySide6'» aunque PyQt5 o PyQt6 está instalado	La aplicación está escrita para PySide6; PyQt no lo reemplaza. Instalar PySide6 con python -m pip install PySide6 (puede convivir con PyQt).
«No module named 'PySide6'» (o pypdfium2, reportlab…)	El entorno virtual no está activado o las bibliotecas se instalaron en otro Python. Activar el entorno y repetir python -m pip install -r requirements.txt. Para ver con qué Python se está trabajando: python -c "import sys; print(sys.executable)".
La instalación de una biblioteca falla	Actualizar pip (python -m pip install --upgrade pip) y reintentar. Comprobar que Python sea de 64 bits y la versión 3.13. Si el equipo usa proxy, configurarlo para pip.
«OCR no disponible»	Falta el programa Tesseract (paso 6) o la biblioteca pytesseract. Instalar y reiniciar la aplicación; si está en otra carpeta, definir TESSERACT_CMD.
«Exportar a Excel» avisa que falta una biblioteca	Instalar openpyxl: python -m pip install openpyxl.
El .exe se cierra o muestra «No module named …»	Falta incluir una biblioteca en el ejecutable. Volver a generarlo agregando --hidden-import con el nombre del módulo que indica el mensaje.
Windows o el antivirus bloquean Inventario.exe	El ejecutable no está firmado digitalmente. Elegir «Más información › Ejecutar de todas formas» o agregar la carpeta a las excepciones del antivirus.
La aplicación abre vacía en otro equipo	Se copió el programa pero no las carpetas Base_Inventario y adjuntos. Copiarlas junto a Inventario.exe.
«database is locked» o error al abrir la base	La base está abierta en otro programa (por ejemplo, DB Browser) o en otra copia de la aplicación. Cerrarlos y reintentar.

15. Nota para Linux y macOS

La aplicación está pensada y probada para Windows. En otros sistemas el procedimiento es análogo, pero estos comandos no se verificaron con la aplicación en este proyecto. El entorno virtual y las bibliotecas de Python se instalan igual, cambiando la activación y el nombre del comando de Python:

```
python3 -m venv venv
source venv/bin/activate
python3 -m pip install -r requirements.txt
python3 app_inventario.py
```

Para el OCR opcional, Tesseract se instala con el gestor de paquetes del sistema:

```
# Debian / Ubuntu
sudo apt install tesseract-ocr tesseract-ocr-spa
# macOS (Homebrew)
brew install tesseract tesseract-lang
```

Referencias

- Python: https://www.python.org/downloads/

- winget, paquete de DB Browser for SQLite: https://winstall.app/apps/DBBrowserForSQLite.DBBrowserForSQLite
- winget, paquete de Tesseract OCR (UB Mannheim): https://winstall.app/apps/UB-Mannheim.TesseractOCR
- Instaladores de Tesseract para Windows: https://github.com/UB-Mannheim/tesseract/wiki
- Parámetros de instalación silenciosa (/S) de Tesseract: https://wingetly.io/apps/ub-mannheim/tesseract-ocr/silent-install
- Datos de idioma de Tesseract: https://github.com/tesseract-ocr/tessdata_fast

Apéndice A. Árboles de directorio: sin ejecutable y con ejecutable en el pendrive

Este apéndice reúne los árboles de carpetas de los tres casos de uso. Las carpetas marcadas «datos» (Base_Inventario, adjuntos y Etiquetas) las crea y mantiene la aplicación sola; son lo que hay que respaldar. Las letras de unidad (C:, E:) son ejemplos.

Caso	Dónde	Necesita Python en el equipo	Se abre con
D.1 Sin ejecutable, en la computadora de desarrollo	C:\Inventario	Sí	python app_inventario.py
D.2 Sin ejecutable, llevado al pendrive	E:\Inventario	Sí (instalado en cada equipo)	python app_inventario.py
D.3 Con ejecutable, en el pendrive	E:\Inventario	No	Inventario.exe (doble clic)

D.1 Sin ejecutable: carpeta de la aplicación en la computadora de desarrollo

Es la carpeta donde se trabaja con Python. Contiene el código, el entorno virtual (opcional) y, después de usar la aplicación, las carpetas de datos.

```
C:\Inventario\
├── app_inventario.py                ← archivo de arranque
├── config.py                        ← rutas de la aplicación
├── catalogo.py                      ← taxonomía y listas iniciales
├── db.py                            ← acceso a SQLite (todo el SQL)
├── almacenamiento.py                ← documentos guardados por hash SHA-256
├── utilidades.py
├── ocr.py                           ← reconocimiento de texto (Tesseract)
├── pdf_motor.py                     ← lectura de PDF (pypdfium2, sin Qt)
├── capital.py                       ← valorización y planilla Excel
├── migrar_a_hash.py                 ← migración de adjuntos antiguos
├── requirements.txt                 ← lista de bibliotecas
├── interfaz\                        ← ventanas y diálogos
│   ├── __init__.py
│   ├── widgets.py
│   ├── arbol_parametros.py
│   ├── ventana_principal.py
│   ├── dialogo_componente.py
│   ├── dialogo_parametros.py
│   ├── dialogo_catalogos.py
│   ├── dialogo_capital.py
│   ├── dialogo_etiquetas.py
│   ├── editor_imagen.py
│   ├── visor_pdf.py
│   └── __pycache__\                 ← la crea Python solo
├── __pycache__\                     ← la crea Python solo
├── venv\                            ← entorno virtual (opcional)
├── Tesseract-OCR\                   ← opcional (OCR portátil)
├── Base_Inventario\                 ← datos
│   └── inventario_electronica.sqlite3
├── adjuntos\                        ← datos
│   ├── almacen\                     (documentos por hash, subcarpetas de 2 caracteres)
│   ├── tmp\
│   └── papelera\
└── Etiquetas\                       ← datos (PDF de etiquetas)
```

D.2 Sin ejecutable, llevado al pendrive

Del caso anterior se copian los archivos de código, la carpeta interfaz, requirements.txt y los datos. No se copian venv ni las carpetas __pycache__: el entorno virtual no es portátil y Python vuelve a crear __pycache__ en cada equipo. En cada computadora donde se use debe haber Python 3.13 con las bibliotecas instaladas (capítulos 2 a 5).

```
E:\Inventario\
├── app_inventario.py
├── config.py
├── catalogo.py
├── db.py
├── almacenamiento.py
```

```
├── utilidades.py
├── ocr.py
├── pdf_motor.py
├── capital.py
├── migrar_a_hash.py
├── requirements.txt
├── interfaz\
│   ├── __init__.py
│   ├── widgets.py, arbol_parametros.py, ventana_principal.py
│   ├── dialogo_componente.py, dialogo_parametros.py, dialogo_catalogos.py
│   └── dialogo_capital.py, dialogo_etiquetas.py, editor_imagen.py, visor_pdf.py
├── Tesseract-OCR\                    ← opcional (OCR portátil)
├── Base_Inventario\                  ← datos
│   └── inventario_electronica.sqlite3
├── adjuntos\                         ← datos
│   ├── almacen\
│   ├── tmp\
│   └── papelera\
└── Etiquetas\                        ← datos

Se abre, en cada equipo, con:   cd E:\Inventario   y   python app_inventario.py
(la letra de unidad puede cambiar de un equipo a otro; la aplicación no depende de ella).
```

D.3 Con ejecutable, en el pendrive

Es el resultado de copiar dist\Inventario al pendrive (capítulos 11 y 12). No hay archivos .py ni entorno virtual: Python y las bibliotecas van dentro de la carpeta _internal. El equipo donde se use no necesita nada instalado.

```
E:\Inventario\
├── Inventario.exe                    ← el programa (doble clic para abrir)
├── _internal\                        ← Python y bibliotecas (no tocar)
│   ├── python313.dll
│   ├── base_library.zip
│   ├── PySide6\ …                    (interfaz gráfica y Qt)
│   ├── pypdfium2\ …                  (visor de PDF)
│   ├── reportlab\ …, PIL\ …, openpyxl\ …
│   └── (otros archivos de apoyo)
├── Tesseract-OCR\                    ← opcional (OCR portátil)
│   ├── tesseract.exe
│   └── tessdata\                     (eng.traineddata, spa.traineddata)
├── Base_Inventario\                  ← datos (la crea el programa la primera vez)
│   └── inventario_electronica.sqlite3
├── adjuntos\                         ← datos
│   ├── almacen\
│   │   ├── 3f\                    (subcarpeta = 2 primeros caracteres del hash)
│   │   │   └── 3fa9c1…e7b2.pdf
│   │   └── a0\
│   │       └── a04d58…19c0.png
│   ├── tmp\
│   └── papelera\
└── Etiquetas\                        ← datos (PDF de etiquetas)
```

Con el ejecutable en el pendrive, el archivo de código fuente no viaja: para modificar la aplicación hay que volver a la computadora de desarrollo (D.1), cambiar el código y generar de nuevo el ejecutable. Para actualizar el pendrive se reemplazan solo Inventario.exe y _internal; las carpetas de datos no se tocan.

D.4 Qué cambia entre los tres casos

Elemento	D.1 Desarrollo	D.2 Pendrive sin .exe	D.3 Pendrive con .exe
Archivos .py y carpeta interfaz	Sí	Sí	No
requirements.txt	Sí	Sí (para instalar en cada equipo)	No
Inventario.exe y _internal	No (están en dist, tras generarlo)	No	Sí
venv (entorno virtual)	Opcional	No (no es portátil)	No
__pycache__	Lo crea Python	No se copia	No existe
build y dist (de PyInstaller)	Solo si se generó el .exe	No	No (se copia el contenido de dist\Inventario)
Tesseract-OCR	Opcional	Opcional	Opcional
Base_Inventario y adjuntos	Sí (datos)	Sí (datos)	Sí (datos)